DP-OPS-0043: SYSTEMIC HARDENING OF THE TASK SUBSYSTEM AND ALGORITHMIC CERTIFICATION OF THE HARVESTER

1. ARCHITECTURAL MANDATE AND GOVERNANCE CONTEXT

The trajectory of the Stela platform's governance architecture has moved decisively toward a model of "mechanical constitutionalism," where the rules of operation are not merely suggested by documentation but enforced by binary code. This shift, necessitated by the increasing complexity of repo-resident artificial intelligence operations, demands that the artifacts governing agent behavior—specifically Tasks and Skills—be treated with the same rigor as compiled software. The drafting of **DP-OPS-0043** represents a critical juncture in this evolution: the application of the rigorous "pointer-first" standards established in **DP-OPS-0041** to the Task subsystem.

The "Policy of Truth" (PoT.md), the repository's constitutional document, establishes a "Filing Doctrine" that strictly segregates operational logic from documentation.[1] Under this doctrine, ops/ is designated for execution ("Run"), while docs/ is reserved for explanation ("Explain"). A critical vulnerability, identified as a "Context Hazard" in PoT.md Section 1.2, arises when library documents—agents, tasks, or skills—are included in the global context manifest (ops/lib/manifests/CONTEXT.md).[1] This hazard implies that if these libraries are loaded indiscriminately, they pollute the agent's context window with stale or irrelevant logic, leading to hallucination and operational drift. Therefore, these artifacts must be strictly **Just-In-Time (JIT)** resources, loaded only when explicitly invoked by a Dispatch Packet (DP).

For a JIT system to function with high integrity, the metadata and provenance of the JIT artifacts must be unimpeachable. An agent loading B-TASK-01 to perform a code review must trust that the task is current, authorized, and linguistically precise. DP-OPS-0041 successfully certified the Skills subsystem by purging narrative prose, enforcing a "registry-as-truth" model, and mandating mechanical harvesting via ops/lib/scripts/skill.sh.[1] DP-OPS-0043 is mandated to mirror this exact rigor for the Task subsystem. The objective is not merely to refactor markdown files but to fundamentally strengthen the ops/lib/scripts/task.sh harvester algorithm to ensure that no task can be born into the repository without a valid genetic marker of its origin—its provenance.

This report provides a comprehensive architectural analysis, a detailed algorithmic

specification for the harvester upgrade, and the complete text of the Dispatch Packet DP-OPS-0043. It synthesizes the logic of PoT.md, the structural contracts of TASK.md, and the heuristic precedents of the heuristics.sh library to define a "serviceable" Task subsystem.

---

2. THEORETICAL FRAMEWORK: THE POINTER-FIRST DOCTRINE

2.1 The Failure of Narrative Documentation

In the nascent stages of autonomous coding systems, governance was often attempted through narrative documentation—long-form prose describing how to perform operations. "To review code, run the linter and check for errors," a task might say. This approach suffers from immediate entropy. As the underlying toolchain evolves—perhaps tools/lint/style.sh is updated to catch unicode contractions [1]—the narrative description in the task file becomes stale. The agent, following the task, might execute the old logic described in the text rather than the new logic enforced by the binary.

This divergence is classified in PoT.md Section 2 as **"Canon Drift,"** a system failure state requiring an immediate stop.[1] To mitigate this, Stela has adopted a **"Pointer-First" Constitution**. Under this doctrine, a task should never describe *how* to perform a standard operation. Instead, it must point to the Single Source of Truth (SSOT) that performs it. A task should not list the forbidden contractions; it should point to tools/lint/style.sh, which mechanically enforces the linguistic standard.

DP-OPS-0041 applied this to Skills, stripping them of redundancy. DP-OPS-0043 extends this to Tasks. The danger in Tasks is acute because they contain **Execution Logic**—sequential steps that an agent performs. If these steps are narrative rather than pointer-based, the agent is operating in a "ghost jurisdiction," following rules that exist only in text and not in code.

2.2 Provenance as the Chain of Custody

In a stateless system where agents spin up and down per session [1], understanding the *origin* of a logic file is as important as its content. Why does B-TASK-05 exist? When was it captured? What operational friction necessitated it?

The concept of **Provenance**, codified in the headers of library files during the DP-OPS-0036 and DP-OPS-0039 upgrades [1], answers these questions. It transforms a static file into a historical record. The ops/lib/scripts/skill.sh harvester introduced automated provenance capture, recording the DP-ID, Branch, HEAD, and Objective at the moment of creation.[1]

DP-OPS-0043 mandates that the Task harvester (ops/lib/scripts/task.sh) must possess the

same capability. It acts as the "Timekeeper" for logic. By integrating with ops/lib/scripts/heuristics.sh [1], the task harvester can detect the "Hot Zone" (the file with the most active churn) and embed that into the task's metadata. This ensures that every task is explicitly linked to the code friction that spawned it, preventing the accumulation of "zombie tasks" that serve no clear purpose.

---

3. AUDIT OF THE CURRENT TASK SUBSYSTEM

3.1 The Registry and the Filing Doctrine

The docs/ops/registry/TASKS.md file serves as the SSOT for all canonical tasks.[1] Under the Filing Doctrine, any markdown file in docs/library/tasks/ that is not listed in this registry is "debris" and must be purged. Conversely, any registry entry that does not resolve to a file is a "dead pointer."

Currently, the registry lists tasks such as B-TASK-01 (Code Review) through B-TASK-06 (Update Documentation).[1] These tasks have undergone preliminary refactoring in DP-OPS-0039 to include "Closeout pointers," but the mechanism for their creation remains susceptible to human error. If an Operator manually copies B-TASK-01 to create B-TASK-07 but forgets to update the provenance block or the closeout step, the system integrity is compromised.

3.2 The "Closeout" Vulnerability

DP-OPS-0039 introduced a critical doctrine: **"Updated task doctrine to require explicit Closeout pointers in Execution Logic"**.[1] This is derived from the PoT.md axiom of "Equilibrium Maintenance"—a task is only complete when the SoP.md ledger is updated to reflect the change.

Currently, this reliance is placed on the human Operator to remember to include "Section 4 Closeout" in the task definition. This is a violation of the **"Mechanical Enforcement"** principle. Governance rules that rely on human memory are destined to fail. DP-OPS-0043 seeks to move this enforcement from the "Review" phase (human) to the "Creation" phase (machine). The harvester algorithm must be hardcoded to inject this closeout step into every draft it generates, making it impossible to create a compliant task without it.

---

4. TECHNICAL SPECIFICATION: THE STRENGTHENED HARVESTER ALGORITHM

The centerpiece of DP-OPS-0043 is the upgrade of ops/lib/scripts/task.sh. This script must

evolve from a template copier into a context-aware generation engine.

4.1 Integration of Heuristic Logic

The harvester must source ops/lib/scripts/heuristics.sh to access the detect_hot_zone and detect_high_churn functions.[1] These functions utilize git to analyze the repository state.

Algorithm 1: Heuristic Context Capture

1. **Input:** The harvester is invoked.
2. **Context Detection:**
   - Call git rev-parse --abbrev-ref HEAD to get the Branch.
   - Call git rev-parse HEAD to get the HEAD hash.
   - Call detect_hot_zone "main" "HEAD" to identify the file with the highest numerical churn (insertions + deletions).[1]
   - Call detect_high_churn to list files with multiple commits in the difference range.
3. **Output:** A structured "Friction Context" string, e.g., "Hot Zone: ops/bin/dump; High Churn: tools/lint/task.sh".

This data is crucial for the "Forensics" skill (S-LEARN-06) [1], enabling future agents to understand *why* a task was created based on the code instability at the time.

4.2 Registry Locking and Collision Avoidance

To enforce the registry as SSOT, the harvester must perform a "Lock Check" before generating any file.

Algorithm 2: Registry Lock

1. **Input:** User provides a proposed ID (e.g., B-TASK-07).
2. **Validation:**
   - Read docs/ops/registry/TASKS.md.
   - Check for existence of | B-TASK-07 |.
3. **Decision Gate:**
   - **IF Found:** STOP. Return Error: "ID Collision: B-TASK-07 is already registered. Use ops/lib/scripts/task.sh list to see available slots."
   - **IF Invalid Format:** STOP. Return Error: "ID must match B-TASK-XX pattern."
   - **IF Valid & Unique:** PROCEED to draft generation.

4.3 The Pointer-First Template Generation

The harvester acts as the factory for TASK files. It must output a markdown structure that strictly adheres to PoT.md.

Algorithm 3: Template Construction

The script uses a heredoc pattern to generate the file content. Key requirements for the

generated text:

- **Header:** Must be # Task: ${NAME}.
- **Provenance Section:** Must be populated with the variables captured in Algorithm 1.
- **Pointers Section:** Must automatically list the "Hard Constraints" from PoT.md Section 4.3: PoT.md, docs/GOVERNANCE.md, TASK.md.[1]
- **Execution Logic:** Must include a hardcoded final step: "N. Closeout: Complete Closeout per TASK.md Section 4."

Pseudo-code Specification:

Bash

```
# ops/lib/scripts/task.sh snippet logic

generate_draft() {
  local id="$1"
  local name="$2"
  local objective="$3"

  # Heuristics
  local hot_zone="$(detect_hot_zone)"
  local timestamp="$(date -u)"

  cat <<EOF
# Task: ${name}

## Provenance
- **Captured:** ${timestamp}
- **DP-ID:** ${CURRENT_DP_ID}
- **Objective:** ${objective}
- **Friction Context:** Hot Zone: ${hot_zone}

## Pointers
- **Constitution:** \`PoT.md\`
- **Governance:** \`docs/GOVERNANCE.md\`
- **Contract:** \`TASK.md\`
- **Registry:** \`docs/ops/registry/TASKS.md\`

## Execution Logic
1. (Pre-flight): Run \`tools/lint/context.sh\`.
2. (Execution):
```

```
3. (Verification): Run \`tools/verify.sh\`.
4. (Closeout): Complete Closeout per \`TASK.md\` Section 4.

## Scope Boundary
- **Allowed:** Execute specific steps defined above.
- **Forbidden:** Do not expand scope beyond ${objective}.
EOF
}
```

This automated injection of the "Closeout" step (Step 4) satisfies the "Equilibrium Maintenance" axiom. By hardcoding it, we ensure that no task can be harvested without the mechanism to close the loop.

---

5. DRAFTING DISPATCH PACKET DP-OPS-0043

The following section presents the full text of the Dispatch Packet to be executed. It follows the decimal heading schema (3.1, 3.2, etc.) defined in TASK.md and validated by tools/lint/dp.sh.[1]

DP-OPS-0043: Task Subsystem Hardening and Harvester Certification

3.1 Freshness Gate (Must Pass Before Work)

- **Base Branch:** main
- **Required Work Branch:** work/task-harvester-hardening-0043
- **Base HEAD:** 5b51900d (Current verified HEAD) [1]
- **Context Check:** Run tools/lint/context.sh to verify ops/lib/manifests/CONTEXT.md.

3.2 Required Context Load (Read Before Doing Anything)

- **Loaded:** PoT.md, SoP.md, TASK.md, ops/lib/scripts/skill.sh (Reference Implementation), tools/lint/task.sh (Target for hardening).
- **Reference:** DP-OPS-0041 (Skills System Overhaul) results in SoP.md.

3.3 Scope and Safety

- **Objective:** Certify the Task subsystem as pointer-first and serviceable by strengthening ops/lib/scripts/task.sh with heuristic provenance harvesting, registry locking, and mandatory "Closeout" enforcement.
- **Constraints:**
  - **No Invention:** Do not create new directories. Use storage/handoff/ for drafts.
  - **Context Hazard:** Do NOT add docs/library/tasks/ to ops/lib/manifests/CONTEXT.md.[1]

- **Linguistic Precision:** No contractions in script comments or output messages.

Target Files Allowlist (Hard Gate)

- ops/lib/scripts/task.sh
- tools/lint/task.sh
- docs/library/TASKS.md (Ledger)
- docs/ops/registry/TASKS.md (Registry)
- SoP.md

3.4 Execution Plan (A-E Canon)

A) Registry Audit and Cleanup

1. **Audit:** Compare docs/ops/registry/TASKS.md against docs/library/tasks/.
2. **Prune:** Identify any "ghost" tasks (files present but not registered) or "dead pointers" (registered but missing files).
3. **Action:** If drift is found, correct the registry to match the filesystem (SSOT).

B) Harvester Algorithmic Upgrade (ops/lib/scripts/task.sh)

1. **Heuristics:** Import ops/lib/scripts/heuristics.sh. Update cmd_harvest to capture detect_hot_zone and detect_high_churn outputs.
2. **Locking:** Implement the check_registry_collision function. It must grep the registry for the requested ID and exit 1 if found.
3. **Template:** Rewrite the cat <<EOF block for draft generation.
   - **Must Include:** ## Provenance with timestamps and friction context.
   - **Must Include:** ## Pointers referencing PoT.md and TASK.md.
   - **Must Include:** ## Execution Logic with the literal line: "Complete Closeout per TASK.md Section 4."
4. **Logging:** Ensure the script appends a log entry to docs/library/TASKS.md under ## Candidate Log.

C) Linter Hardening (tools/lint/task.sh)

1. **Closeout Gate:** Add a regex check to lint_task(). It must scan the ## Execution Logic section and fail if the pattern Closeout.*TASK.md is not found in the final steps.
2. **Provenance Gate:** Add a check to ensure DP-ID and Objective in the header are not generic placeholders (e.g., "ENTER_DP_ID") *after* the file is in docs/library/tasks/. (Drafts in storage/ are exempt).

D) Certification Cycle (The "Drift Injection" Test)

1. **Harvest:** Run ./ops/lib/scripts/task.sh harvest --id B-TASK-99 --name "Drift Test" --objective "Testing linter".
2. **Inject Drift:** Manually edit the generated draft in storage/handoff/ to remove the "Closeout" step.

3. **Promote:** Run ./ops/lib/scripts/task.sh promote <draft_path>.
4. **Verify Linter:** Run ./tools/lint/task.sh.
    - **Expected Result: FAIL**. The linter must catch the missing closeout step.
5. **Remediate:** Restore the closeout step and re-run lint.
    - **Expected Result: PASS**.
6. **Cleanup:** Delete B-TASK-99 and revert the registry change.

E) Final Verification

1. Run tools/verify.sh to check filing doctrine.
2. Run tools/lint/style.sh to check for contractions.
3. Run tools/lint/context.sh to ensure no context hazards.
4. Run tools/lint/truth.sh to validate canon integrity.

4 Closeout (Mandatory)

- **Requirement:** Populate storage/handoff/DP-OPS-0043-RESULTS.md with the output of the Certification Cycle (Section D).
- **Thread Transition:** Archive current thread state if successful.
- **SoP Entry:** "DP-OPS-0043 Task Subsystem Hardening. Strengthened harvester with heuristic provenance and registry locking. Hardened linter for closeout enforcement."

---

6. ENFORCEMENT AND VERIFICATION STRATEGY

The success of DP-OPS-0043 relies on the "Immune System" provided by the linters. The modifications to tools/lint/task.sh act as the primary enforcement mechanism.

6.1 The Linter as Gatekeeper

The tools/lint/task.sh script currently checks for basic structure.[1] DP-OPS-0043 mandates expanding its regex capabilities.

Table 1: Enhanced Linter Rules

Target Section	Rule Description	Failure Condition
## Execution Logic	**Closeout Enforcement:** Must verify the presence of the closeout pointer.	FAIL if "Closeout" AND "TASK.md" are not found in the same line within the section.
## Provenance	**Placeholders:** Checks for	FAIL if "ENTER_" or "REPLACE_" tokens exist in

	leftover template vars.	a promoted file.
## Pointers	**Canon Integrity:** Checks for required canon links.	FAIL if PoT.md or TASK.md are missing from the pointer list.

6.2 Context Hazard Verification

The PoT.md strictly prohibits "Context Hazard".[1] The harvester logic must inherently respect this by writing drafts to storage/handoff/ (which is ignored by context.sh checks) and promoting to docs/library/tasks/ (which is explicitly excluded from the global context manifest). The tools/lint/context.sh script [1] already enforces this exclusion; DP-OPS-0043 relies on this existing gate but verifies it via the execution plan.

---

7. OPERATIONAL IMPLICATIONS

The implementation of DP-OPS-0043 fundamentally alters the workflow for creating new operational logic.

1. **Speed vs. Rigor:** The new harvester adds friction to the creation process. An Operator cannot simply "write a task." They must "harvest a task." This friction is intentional. It forces the Operator to define the *objective* and *context* before the logic exists, adhering to the "Provenance-Forward" philosophy defined in README.md.[1]
2. **Auditability:** Future audits of the system will be able to trace every task back to a specific Dispatch Packet and code state (Hot Zone). If B-TASK-05 fails in production, the provenance block allows the engineer to see that it was created when tools/verify.sh was the Hot Zone, providing critical context for debugging.
3. **Stability:** By locking the registry and enforcing unique IDs, we prevent "Split Brain" scenarios where two tasks claim the same identity, a fatal error for autonomous agents resolving dependencies.

8. CONCLUSION

DP-OPS-0043 is the necessary completion of the governance overhaul initiated in DP-OPS-0041. It creates parity between the Skills and Task subsystems, ensuring that the "Active Work Surface" (TASK.md) is supported by a library of tasks that are mechanically certified, strictly provenanced, and linguistically precise.

The strengthened harvester removes human variability from the equation. It embeds the "Policy of Truth" into the very file structure of the task, ensuring that compliance is not an

afterthought, but a prerequisite for existence. This report confirms that the drafted Dispatch Packet meets all criteria of PoT.md and provides a robust blueprint for execution.

Status: Ready for Dispatch.

Next Step: Execute ops/bin/open with intent DP-OPS-0043.

Works cited

1. dump-full-main-5b51900d.txt